ml-evaluation — evaluating ML regional-climate emulators

`ml-evaluation` implements the evaluation indices and performance diagnostics recommended by CORDEX-ML-Bench for assessing machine-learning RCM (regional climate model) emulators.

Two core concepts

- **Indices** describe *one* simulation `x` — its mean, a percentile, wet-day frequency, an annual maximum, and so on.
- **Diagnostics** compare *two* simulations: the reference `x0` (ground truth, the RCM target) against the prediction `x1` (the emulator output) — e.g. mean bias, RMSE, correlation, RALSD.

Everything is exposed through the **`ml-eval`** command-line tool, which offers three tracks:

track	cost	what it computes
`fast`	cheap	mean bias, RMSE, correlations, RALSD, percentile / extreme / precip biases
`slow`	heavy	power spectral density, Wasserstein distance, spell / threshold diagnostics
`multivar`	—	multivariate (e.g. temperature–precipitation) correlation

This guide focuses on the **`fast`** track, because that is what the `run_evaluation_case.sh` driver shown below calls.

> **Getting `ml-eval`.** Install the package first — with **pixi** (`pixi install`) or **conda** (`conda env create -f requirements/environment.yaml` then `pip install -e .`). Confirm it is on the PATH with `ml-eval --help`. See the repository README for the full setup.
>
> **Note.** The commands below are **illustrative** — they reference real paths on our HPC project and are shown for reference, not run from this document. Adapt the paths (or run `run_evaluation_case.sh`) on the machine that holds the data.

Directory layout

The example uses the directory convention produced by **deep4production** on our HPC project `C3S2_384`. **This layout is only a convention** — `ml-eval` itself is *not* tied to it: a single call needs only a reference file (`--x0`), a prediction file (`--x1`), a variable (`--var`) and an output location (`--output`). Point them wherever you like.

The subdirectories `run_evaluation_case.sh` reads from and writes to:

```
C3S2_384/TROPICAL-BENCH/                              <- PROJECT
├── EMULATORS/
│   └── <EMULATOR>/                                   e.g. UNET-SINGLE
│       └── outputs/
│           ├── predictions/
│           │   └── <CHECKPOINT>_<GCM>_<PERIOD>_<CASE>.nc      <- x1  (prediction, INPUT)
│           └── evaluation/                                    <- ml-eval writes HERE
│               └── <var>/
│                   └── <CHECKPOINT>_<GCM>_<PERIOD>_<CASE>/
│                       └── <season>/
│                           ├── metrics_fast_field.zarr
│                           ├── metrics_fast_series.zarr
│                           ├── metrics_fast_float.zarr
│                           ├── metrics_pr_field.zarr          (only when var == pr)
│                           └── metrics_psd.zarr
├── EVALUATION/
│   ├── run_evaluation.sh          (sweep: every model × case × var × season)
│   ├── run_evaluation_case.sh     (single cell — used in this guide)
│   └── plot_hexagon.py            (reads the zarrs, draws the score hexagon)
```

```
└── SOURCE_FILES/
    └── test/
        ├── historical/target/targets_<GCM>_1981-2000.nc      <- x0  (reference)
        └── mid-century/target/targets_<GCM>_2041-2060.nc     <- x0  (reference)
```

- **Predictions (x1)** live under `EMULATORS/<EMULATOR>/outputs/predictions/`, named `<CHECKPOINT>_<GCM>_<PERIOD>_<CASE>.nc`.
- **References (x0)** live under `SOURCE_FILES/test/<period>/target/` (`historical` for 1981-2000, `mid-century` for 2041-2060).
- **Evaluation outputs** land under `EMULATORS/<EMULATOR>/outputs/evaluation/<var>/<stem>/<season>/` — one folder of zarr stores per (variable, prediction, season).

What `ml-eval fast` writes

Each `ml-eval fast --output <DIR>` call fills `<DIR>` with the zarr stores below. Within every store the metrics are stacked as a single `value` variable indexed by a **`diagnostic`** coordinate, so a metric is selected with `ds["value"].sel(diagnostic="<name>")`.

store	`value` dims	`diagnostic` entries	written
`metrics_fast_field.zarr`	(diagnostic, y, x)	`bias_mean_field`, `rmse_field`, `bias_p99_field`, `bias_p1_field`, `bias_acorr_field`, `bias_max1day_field`, `bias_min1day_field`	always
`metrics_fast_series.zarr`	(diagnostic, time)	`rmse_series`, `pcorr_series`, `scorr_series`	always
`metrics_fast_float.zarr`	(diagnostic,)	`rmse_float`, `ralsd_float`	always
`metrics_pr_field.zarr`	(diagnostic, y, x)	`bias_p98wet_field`, `bias_p90wet_field`, `bias_r1_field`, `bias_r10_field`, `bias_sdii_field`, `bias_cdd_field`, `bias_cwd_field`, `relbias_rx1day_field`, `relbias_sdii_field`, `relbias_p98wet_field`, `relbias_p90wet_field`	`--var pr` only
`metrics_psd.zarr`	(diagnostic, wavenumber)	`psd_x0`, `psd_x1`	always

`*_field` stores hold spatial maps (one value per grid point); `*_series` hold a time series (reduced over lat/lon); `*_float` hold domain scalars; `metrics_psd` holds the radially-averaged power spectrum of the reference and the prediction.

Example: `run_evaluation_case.sh`

`run_evaluation_case.sh` evaluates **one** cell — a single (emulator, checkpoint, driving-GCM, period, boundary case, season) — looping over the requested variables. Edit the `PARAMETERS` block and run it (`bash` or `sbatch`). It is the atom that `run_evaluation.sh` loops over the full model roster with.

The script as it currently stands:

```
#!/bin/bash
# ============================================================================
# Single-cell evaluation for TROPICAL-BENCH: one `ml-eval fast` call for one
# (emulator, checkpoint, driving-GCM, period, boundary, variable, season) --
# the atom that run_evaluation.sh loops over. Use it to (re)compute a single
# hexagon cell without sweeping every model/case/variable.
#
# Not a CLI: edit the variables in the PARAMETERS block below, then run the
# script (bash or sbatch). Each variable identifies one cell:
#   EMULATOR     emulator dir under EMULATORS/           e.g. RESDIFF_V3
#   CHECKPOINT   prediction-file prefix (usually the     e.g. RESDIFF_V3
#                run_ID; for multi-checkpoint runs a
#                tag such as RESDIFFbest / RESDIFF500)
#   DRIVING_GCM  driving GCM in the file name            e.g. EC-Earth3
#   PERIOD       <startYYYY>-<endYYYY>                   e.g. 2041-2060
#   CASE         boundary: perfect | imperfect           e.g. perfect
```

```
#   VAR          variable to score                        e.g. pr
#   SEASON       annual | DJF | MAM | JJA | SON | <MON>  (annual => whole year)
#
# From those it builds the prediction file (x1)
#   EMULATORS/<EMULATOR>/outputs/predictions/<CHECKPOINT>_<DRIVING_GCM>_<PERIOD>_<CASE>.nc
# the reference target (x0)
#   SOURCE_FILES/test/<period-dir>/target/targets_<DRIVING_GCM>_<PERIOD>.nc
# and writes the 6 "hexagon" metrics as zarr bundles under
#   EMULATORS/<EMULATOR>/outputs/evaluation/<VAR>/<CHECKPOINT>_<DRIVING_GCM>_<PERIOD>_<CASE>/<SEASO
# -- the same tree run_evaluation.sh uses, so the two are interchangeable and
# plot_hexagon.py finds either.
#
# Run:   bash   EVALUATION/run_evaluation_case.sh      (after editing the vars)
#        sbatch EVALUATION/run_evaluation_case.sh
# ============================================================================
#SBATCH --job-name=eval_case
#SBATCH --output=./logs/eval_case_%j.out
#SBATCH --error=./logs/eval_case_%j.err
#SBATCH --partition=wncompute_meteo
#SBATCH --time=12:00:00
#SBATCH --mem=0


# --- Environment (must contain the ml-evaluation package: `ml-eval`) --------
source /gpfs/projects/meteo/WORK/banoj/miniconda3/etc/profile.d/conda.sh
conda activate d4p

# ---------------------------- DIRECTORIES ----------------------------------
PROJECT=/gpfs/projects/meteo/WORK/PROYECTOS/C3S2_384/TROPICAL-BENCH
cd "$PROJECT"
EMU_DIR=$PROJECT/EMULATORS
EVAL_DIR=$PROJECT/EVALUATION
mkdir -p "$EVAL_DIR/logs"

# ---------------------------- PARAMETERS -----------------------------------
EMULATOR="UNET-SINGLE"
CHECKPOINT="UNET-SINGLE"
DRIVING_GCM="EC-Earth3"
CASE="perfect"
PERIOD="2041-2060"
VARS=("pr" "tas" "tasmax" "tasmin" "hurs" "psl" "uas" "vas")
SEASON="annual"
OVERWRITE=0


# ---------------------------- RESOLVE PATHS --------------------------------
# ---- PREDICTION FILE (x1) ----
STEM="${CHECKPOINT}_${DRIVING_GCM}_${PERIOD}_${CASE}"
PRED="$EMU_DIR/$EMULATOR/outputs/predictions/$STEM.nc"

# ---- REFERENCE FILE (x0) ----
period_dir() { case "$1" in 1981-2000) echo historical ;; 2041-2060) echo mid-century ;; *) echo mi
REF="$PROJECT/SOURCE_FILES/test/$(period_dir "$PERIOD")/target/targets_${DRIVING_GCM}_${PERIOD}.nc"

for var in "${VARS[@]}"; do
    # ---- OUTPUT DIRECTORY ----
    OUT="$EMU_DIR/$EMULATOR/outputs/evaluation/$var/$STEM/$SEASON"
```

```
    mkdir -p "$OUT"
    [ "$SEASON" = "annual" ] && SEASON_FLAG="" || SEASON_FLAG="--season $SEASON"

    # ---------------------------- RUN ------------------------------------------
    echo "[eval] $EMULATOR | $CHECKPOINT | $var | $PERIOD | $SEASON"
    echo "       x0 (ref)  = $REF"
    echo "       x1 (pred) = $PRED"
    echo "       output    = $OUT"
    ml-eval fast --x0 "$REF" --x1 "$PRED" --var "$var" --output "$OUT" $SEASON_FLAG
done
```

What the script wires up (per variable):

- `STEM` = `<CHECKPOINT>_<DRIVING_GCM>_<PERIOD>_<CASE>` → the prediction file `EMULATORS/<EMULATOR>/outputs` (the `--x1`).
- `period_dir` maps the period to the `SOURCE_FILES/test/<...>` subfolder, giving the reference `targets_<DRIVING_GCM>_<PERIOD>.nc` (the `--x0`).
- `OUT = .../outputs/evaluation/<var>/<STEM>/<SEASON>` → the `--output` directory.
- `SEASON_FLAG` is empty for `annual` (no `--season` → whole year) and `--season <S>` otherwise.

Stripped to a single call for one variable, that is just:

```
ml-eval fast \
    --x0 SOURCE_FILES/test/mid-century/target/targets_EC-Earth3_2041-2060.nc \
    --x1 EMULATORS/UNET-SINGLE/outputs/predictions/UNET-SINGLE_EC-Earth3_2041-2060_perfect.nc \
    --var pr \
    --output EMULATORS/UNET-SINGLE/outputs/evaluation/pr/UNET-SINGLE_EC-Earth3_2041-2060_perfect/ar
```

Reading the results back

The stores are plain zarr. Open one, inspect its `diagnostic` coordinate, and select the metric you want. Spatial `*_field` metrics are reduced to a single domain number as `mean(|field|)` — the same reduction `plot_hexagon.py` uses to place a model on each hexagon axis.

```
import os
import numpy as np
import xarray as xr

# One evaluation cell produced by run_evaluation_case.sh
EMU      = "/gpfs/projects/meteo/WORK/PROYECTOS/C3S2_384/TROPICAL-BENCH/EMULATORS"
EMULATOR = "UNET-SINGLE"
STEM     = "UNET-SINGLE_EC-Earth3_2041-2060_perfect"   # <CHECKPOINT>_<GCM>_<PERIOD>_<CASE>
VAR      = "pr"
SEASON   = "annual"

cell = os.path.join(EMU, EMULATOR, "outputs", "evaluation", VAR, STEM, SEASON)

# 1) scalars: rmse_float, ralsd_float
floats = xr.open_zarr(os.path.join(cell, "metrics_fast_float.zarr"))["value"]
print(floats["diagnostic"].values)                        # ['rmse_float' 'ralsd_float']
rmse  = float(floats.sel(diagnostic="rmse_float"))
ralsd = float(floats.sel(diagnostic="ralsd_float"))

# 2) a spatial bias field, reduced to a domain scalar the plot_hexagon way
fields = xr.open_zarr(os.path.join(cell, "metrics_fast_field.zarr"))["value"]
print(fields["diagnostic"].values)
bias_mean_map    = fields.sel(diagnostic="bias_mean_field")   # (y, x) map
bias_mean_scalar = float(np.abs(bias_mean_map).mean())        # mean(|bias|)

# 3) precip-only diagnostics (present because VAR == "pr")
```

```
pr = xr.open_zarr(os.path.join(cell, "metrics_pr_field.zarr"))["value"]
print(pr["diagnostic"].values)
sdii_bias = float(np.abs(pr.sel(diagnostic="bias_sdii_field")).mean())

print(f"rmse={rmse:.4g}  ralsd={ralsd:.4g}  "
      f"|bias_mean|={bias_mean_scalar:.4g}  |bias_sdii|={sdii_bias:.4g}")
```